Agenda

- JSX
- React Component
- Props
- State
- Adding Tailwind
- Signin
- ~~Signup~~

Key Improvements in JSX

1. JSX Syntax: HTML-like templating

```
// Transforms to:
// React.createElement("h1", null, "Hello React")
<h1>Hello React</h1>
```

2. Component Model: Reusable UI units

```
function MyComponent() {
    return <div>Content</div>;
}
```

3. Readability: Visual hierarchy matches DOM structure

4. Developer Experience: Faster prototyping

React Important Concepts

1. Component Architecture

- Reusable Components

```
// Component = JavaScript function returning JSX
function Greeting() {
    return <h2>Welcome to React!</h2>;
}

// Component Composition
function App() {
    return (
        <div>
            <h1>Hello World</h1>
            <Greeting />
            <Greeting />
```

SUNBEAM INFOTECH

```
        </div>
    );
}
```

2. JSX Styling

- Inline styles as objects

```
<div className="container" style={{ color: "red" }}>
    <p>Styled content</p>
</div>
```

```
const style = {
    container: {
        color: "red"
    },
    content: {
        fontSize: "14px",
        color: "blue"
    }
}

<div className="container" style={style.container}>
    <p style={style.content}>Styled content</p>
</div>
```

- class vs className

```
// Correct
<div className="book-card">...</div>

// Incorrect
<div class="book-card">...</div>
```

3. Embedding JavaScript in JSX/Data binding

- Using the variable value inside a html tag
- In react, it will be done using interpolation
- use the {} brackets for loading the variables value inside the html tag

```
const myvar = 100
<h3>{myvar}</h3>
```

SUNBEAM INFOTECH

- To render an object, split the object into properties and use interpolation to render those properties

```
const jamesBond = {
    name: "James Bond",
    age: 65,
    address: "London",
    speciality: "Adventurous Spy",
};

// Variables in curly braces
<div>Name: {jamesBond.name}</div>
<div>Address: {jamesBond['address']}</div>

// Expressions work too
<div>Status: {age > 60 ? 'Veteran' : 'Active'}</div>
```

- to render an array, use the map function to iterate over the array and use interpolation to render the properties of the object

```
const heros = [
    {
        name: "Iron Man",
        age: 48,
        address: "Stark Tower, New York",
        speciality:
            "Genius-level intellect, Powered armor suit, Advanced technology",
    },
    {
        name: "Captain America",
        age: 105,
        address: "Brooklyn, New York",
        speciality: "Super strength, Enhanced agility, Vibranium shield combat",
    },
];

const MarvelHeros = () => {
    return (
        <div>
            {heros.map((hero) => (
                <div>
                    <h3>Marvel Hero</h3>
                    <div>Name: {hero.name}</div>
                    <div>Address: {hero.address}</div>
                    <div>Age: {hero.age}</div>
                    <div>Speciality: {hero.speciality}</div>
                </div>
            ))}
        </div>
    );
```

SUNBEAM INFOTECH

```
};
export default MarvelHeros;
```

JSX Syntax understanding for

1. Self-Closing Tags

```
// Invalid
<img src="..." alt="..." width="60px">

// Always required for void elements
<img src="..." alt="..." width="60px" />
```

2. Root Element Requirement

```
// Invalid
return (
  <h3>Title</h3>
  <p>Content</p>
)

// Valid
return (
  <div>
    <h3>Title</h3>
    <p>Content</p>
  </div>
)

// Valid (Non rendered - parent tag)
return (
  <>
    <h3>Title</h3>
    <p>Content</p>
  </>
)
```

SUNBEAM INFOTECH

Summary: conventions to write JSX

- use xml (html) syntax in JS
- the properties are using camelCase: className
- to read value of a variable use string interpolation ({})
- array
    - to read values of an array use map function
    - in array, every element must have a unique key to identify each element uniquely

    - these keys are used by diffing algorithm to track the changes from older to news state of virtual DOM
- execute the JS expression or function using {}

React Component

- Component is reusable entity which contains user interface defined in html code
- A component can be loaded using the component name as a tag (enclosed by < and >)
- It Contains -

1. user interface: created using JSX syntax
2. state (optional): used to store some data for internal use
3. event handlers (optional): used to handle events generated by the component UI
4. props (optional): parameters (properties) of a component
5. mandatory to return only one element as a return value of a component. If you have multiple elements, then wrap them inside a parent element

- Types of components

1. Functional component

- Component created using a function
- A javascript function which returns a JSX code to create its user interface
- Before react 16, functional components were used only for stateless implementation (the component which does not require to maintain the state)
- stateless means a component which is not maintaining its state
- Since the react 16 introduced a concept called as a react hooks, it is possible now to create functional components to store the state
- Hence the class components are not need anymore and by default we use a function to create component
- it is faster than class component
- it is light weight component

```
const root = document.getElementById('root')
// functional component
function Dummy() {
    return <div>this is a functional component </div>
}
// here the <Dummy /> is function component
ReactDOM.createRoot(root).render(<Dummy />)
```

2. Class component

- Component create using a class
- Before react 16, class components were used to create stateful components (a component which can maintain its state)
- It is a class derived from React.Component class

SUNBEAM INFOTECH

- this class implements method named render()
- used a lot before react hooks to create stateful component
- stateful means a component which is maintaining its state for internal use
- using class component is not required as the react hook can maintain the state inside a functional component

```
const root = document.getElementById('root')
// class component
class Dummy extends React.Component {
  render() {
    return <div>this is a class component </div>
  }
}
// here the <Dummy /> is class component
ReactDOM.createRoot(root).render(<Dummy />)
```

Component Event Handling

- Components should handle its events to make the component easily reusable.
- Event handling in React follows camelCase naming convention (`onClick` instead of `onclick`)
- Uses Synthetic Events - React's cross-browser wrapper around native DOM events
- Event handlers receive a SyntheticEvent object containing event details

Basic Event Handling

- define a event handler inside a component
- event handler
  - function which handles a required event
  - e.g. onButtonClick() => click event of a button
- use event names provided by react
  - e.g. click => onClick

```
function App() {
    // 1. Define handler function
    const handleButtonClick = () => {
        alert("Button clicked!");
    };

    return (
        <div>
            {/* 2. Pass function reference (no parentheses!) */}
            <button onClick={handleButtonClick}>Click Me</button>
        </div>
    );
}
```

- Pass function reference - `onClick={handleClick}`

SUNBEAM INFOTECH

- Avoid function call - `onClick={handleClick()}` would execute immediately
- SyntheticEvent provides consistent interface across browsers

Handle user inputs

```
function App() {
    const handleInputChange = (event) => {
        // Access input value
        const userInput = event.target.value;
        console.log(`User input: ${userInput}`); // Correct string interpolation
    };
    return (
        <div>
            <label>Username:</label>
            <input
                type="text"
                onChange={handleInputChange}
                placeholder="Type something..."
            />
        </div>
    );
}
```

- Use `event.target.value` for input elements
- Always use proper string interpolation with backticks: ``` `Value: ${value}` ```

Event Handler Best Practices

1. Naming Convention : Use `handle[Element][Event]` pattern

```
handleButtonClick, handleInputChange;
```

2. Inline Handlers (for simple logic):

```
<button onClick={() => console.log("Clicked")}>Log Click</button>
```

3. Parameter Passing:

```
<button onClick={(e) => handleClick(id, e)}>Delete Item</button>
```

Props (Parent->Child Data Flow)

- It is an object which will be passed from a parent to child component
- It is a parameter to the component function

SUNBEAM INFOTECH

- props are readonly i.e the child must not modify the data sent by the parent component (even if the child modifies the props, the parent wont be able to see the changes)

```
// child
function Person(props) {
  return (
    <div>
      <h3>Person</h3>

      {/* reading the data sent by the parent component */}
      <div>name = {props.name}</div>
    </div>
  )
}

// child
// props gets destructured into model and company variables
function Car({ model, company }) {
  return (
    <div>
      <h3>Car</h3>

      {/* reading the data sent by the parent component */}
      <div>model = {model}</div>
      <div>company = {company}</div>
    </div>
  )
}

// parent
function App() {
  const name = 'person1'
  const car = { model: 'triber', company: 'renault' }
  return (
    <div>
      <h2>Welcome to react</h2>
      {/* the parent is passing the data to the child component */}
      <Person name={name} />
      <Car
        model={car['model']}
        company={car['company']}
      />
    </div>
  )
}
```

SUNBEAM INFOTECH

Single Responsibility Principle

- when data is passed from parent component to child component using props it is recommended for the components to follow single responsibility principle

- In this priniple it is recommended to pass the properties of object individually rather than passing the entire object.
- Passing a large object can lead to components having access to data they don't need, potentially encouraging them to take on responsibilities outside their intended scope.
- Increased Coupling: Passing entire objects can create tighter coupling between parent and child components. If the structure of the object changes, many child components might need to be updated, even if they only use a small part of the object. Passing only necessary properties reduces this coupling.

```
// Good: Focused component
function BookPrice({ price }) {
    return <div>Price: Rs. {price}/-</div>;
}

// Bad: Mixed responsibilities
function BookDetails({ book }) {
    // Handles price, image, author...
}
```

React Hook

- a JS function defined by react which is used to perform some logic on the components
- These are special functions prefixed with `use` that let you "hook into" React features
- react introduced the react hooks in version 16.0
- e.g.
    - built-in hooks
        - useState()
        - useContext()
        - useEffect()
        - useReducer()
        - useCallback()
        - useMemo()
        - useRef()
        - useId()
    - react router hooks
        - useNavigate()
        - useParams()
        - useLocation()
    - redux hooks
        - useDispatch()
        - useSelector()

Rules to call React Hooks

- Never call inside loops, conditions, or nested functions
- Ensures consistent hook order between renders
- Only Call Hooks from React Functions like
    1. Functional components

SUNBEAM INFOTECH

2. Other custom hooks

State

- object which represents the component's inner data
- you may use one or more state members
- when there is a change in the state (members), the component rerenders the application with updated values
- state is a read writable object
- to create a state inside the component
  - inside functional component: useState()
  - inside class component: class components already has gotten a property (state) with all the properties and their values
- every component creates its own state
- unlike the props, the state is not meant for sharing the data with the child

useState() Hook

- Adds reactive state to functional components
- It returns an array with two members
- Returns a stateful value at arr[0] and a function at arr[1] to update it
- It is used to maintain the component's state
- when the state get updated, the component gets re-rendered
- accepts only one parameter to initialize the state member
- first member array[0]: reference to the value inside the state (getter)
- second member array[1]: reference to the function used to update the value (setter)

- Syntax

```
const [state, setState] = useState(initialValue);
```

Part	Purpose
`state`	Current state value (read-only)
`setState`	Updater function (triggers re-render)
`initialValue`	Initial state (can be any type - number, string, object, etc.)

- Example

```
function Counter() {
  const [counter, setCounter] = useState(0)

  const onIncrement = () => {
    // trigger the component rerendering
    setCounter(counter + 1)
```

SUNBEAM INFOTECH

```
    }

    const onDecrement = () => {
      // trigger the component rerendering
      setCounter(counter - 1)
    }

    return (
      <div>
        <h3>Counter</h3>

        <div>counter = {counter}</div>
        <button onClick={onIncrement}>Increment</button>
        <button onClick={onDecrement}>Decrement</button>
      </div>
    )
  }
  export default Counter
```

component rerender

- invoking the component with required features with updated value
- to trigger rerendering update the component state

SUNBEAM INFOTECH